

CONECTEA

Documento Central Obrigatório

Regras operacionais, DS V2, segurança, modularidade e fechamento por escopo

Versão 4.3 - Fonte obrigatória viva do Projeto ConeCTEA

Produto final em construção - não tratar como MVP.

Aplicação: qualquer tela, fluxo, feature, service, RPC, migration, Edge Function, visual, dado, documento, integração, DS V2, Campo Cadastral, helper padronizado ou programa terceirizado incorporado ao projeto.

Regra central: auditar, classificar, modularizar, proteger, validar, documentar a evolução permanente e só então avançar.

Regra financeira: ConeCTEA deve operar com custo zero; recurso pago não entra sem autorização explícita e caixa definido.

Este documento atualiza a versão 4.2 e preserva suas regras obrigatórias. A v4.3 reforça a governança de documento vivo, atualiza o inventário da Camada Padronizada V2/DS V2 e explicita que novas capacidades permanentes adicionadas durante escopos aprovados devem entrar neste documento ao fechamento da frente.

Fixar como fonte principal do Projeto ConeCTEA e revisar em todo chat novo antes de prompts ou decisões sensíveis. Quando houver conflito, prevalece a regra mais segura, mais restritiva, mais gratuita, mais acessível e o código real atual.

Uso interno - Família TEA Bauru / ConeCTEA. Documento vivo e versionado.

Índice de consulta rápida

- 1. Regra máxima e aplicação por escopo
- 2. Produto, custo zero e limites financeiros
- 3. Papéis de Lucas, ChatGPT, Antigravity e Codex
- 4. Ordem obrigatória dentro de cada escopo
- 5. Auditoria, relatório e evidências
- 6. Camada Padronizada V2, DS V2 viva e satélites oficiais
- 7. Legado, lacuna e regra de migração
- 8. Três camadas visuais: local, compartilhada e global
- 9. Semântica visual, DsCores e DsTokenStatus
- 10. UI/UX neurodivergente e responsividade global
- 11. Arquitetura modular por features
- 12. Segurança, LGPD, Play Console e dados sensíveis
- 13. Supabase, CRUD, RLS, GRANTS, RPCs e Edge Functions
- 14. Fluidez local e otimização de egress
- 15. Documentação, validação, commit e checklist final
- 16. GitHub Desktop: commit e push manuais
- 17. Governança de atualização do documento central
- 18. Frases de decisão para novos chats
- 19. Resumo executivo obrigatório
- 20. Fontes consolidadas, inventário v4.3 e histórico

Regra de leitura obrigatória: cada nova frente deve ser tratada como um novo escopo. Ao terminar um escopo, executar a auditoria final e o checklist. Se o escopo criou, removeu, consolidou ou reclassificou regra permanente, atualizar este Documento Central em nova versão antes de considerar a frente institucionalmente fechada.

1. Regra máxima e aplicação por escopo

O ConeCTEA deve ser tratado como produto final em construção. Ele pode evoluir por etapas, mas nenhuma etapa deve nascer como improviso, MVP descartável ou código temporário sem controle.

Este documento se aplica a qualquer tela, fluxo, feature, widget, helper, service, model, route, migration, RPC, Edge Function, integração, documento, prompt, validação, correção visual, refino de responsividade, otimização, segurança ou preparação para Play Console.

Regra central: dentro de cada escopo, entender, auditar, classificar, modularizar quando necessário, proteger dados, respeitar DS V2 e satélites oficiais, validar responsividade, checar segurança, otimizar fluidez e egress, relatar evidências, atualizar documentação permanente quando aplicável e só então avançar.

1.1 Escopo e subescopo

Escopo é a frente autorizada por Lucas no chat atual. Pode ser uma tela, uma parte de tela, um fluxo de CPF, uma frente de notificações, uma revisão de admin, uma evolução de DS V2, uma migration, uma auditoria ou uma atualização documental.

Subescopo é uma parte menor dentro da frente. Exemplo: dentro do fluxo de CPF, pode haver subescopo visual, subescopo de RPC, subescopo de GRANT, subescopo de validação local e subescopo de Play Console/LGPD.

Cada escopo e subescopo deve respeitar este documento. Fechar um escopo não autoriza pular auditoria no próximo.

1.2 O que significa pronto

Uma frente não está pronta porque compila, porque a tela abre ou porque o executor escreveu 'feito com sucesso'. Pronto significa: escopo cumprido, nenhuma alteração fora do pedido, dados protegidos, DS V2 respeitada, responsividade considerada, validações executadas, riscos classificados, relatório factual, validação manual/visual quando houver UI e documentação viva atualizada quando a frente criou regra permanente.

2. Produto, custo zero e limites financeiros

Regra financeira fundamental: o ConeCTEA deve operar com custo financeiro zero. O caixa do projeto é zero. Nenhuma solução, API, serviço, banco, envio de e-mail, notificação, armazenamento, analytics, OCR, IA, mapa, SMS, hospedagem, automação ou integração pode depender de pagamento para funcionar.

Sempre que uma frente exigir recurso novo, a primeira decisão técnica deve ser buscar uma opção gratuita que atenda ao ConeCTEA. Se não existir, a segunda decisão deve ser desenhar uma alternativa gratuita, reduzir escopo, mudar estratégia ou adiar a funcionalidade.

2.1 Proibições de custo

- Não contratar serviço pago, API paga, add-on pago, plano Pro, assinatura, pacote de credits, SMS pago, e-mail pago, OCR pago, IA paga, storage pago ou automação paga sem autorização explícita de Lucas e existência real de caixa.
- Não criar dependência que funcione apenas em trial, limite promocional ou cota temporária.
- Não implementar funcionalidade que pare de funcionar quando o limite gratuito acabar sem plano de contingência.
- Não resolver problema gratuito atual empurrando para plataforma paga.
- Não esconder custo indireto: egress, storage, invocação, imagem, log, build, webhook, fila, SMTP, push, analytics ou retenção também contam.

2.2 Checklist de custo antes de recurso novo

- ☐ O recurso é gratuito hoje?
- ☐ O plano gratuito cobre o uso esperado do ConeCTEA?
- ☐ Existe custo por envio, chamada, usuário, arquivo, egress, storage ou invocação?
- ☐ Existe alternativa gratuita por Google Apps Script, Supabase Free, OneSignal Free, recurso nativo ou fluxo manual?
- ☐ A funcionalidade pode ser redesenhada para custo zero?
- ☐ O relatório informou risco de custo futuro ou limite de cota?
- ☐ Lucas autorizou explicitamente qualquer exceção financeira?

Se uma única resposta indicar custo ou risco financeiro não resolvido, a frente deve parar para decisão de produto. A regra é criar o ConeCTEA de forma sustentável dentro do custo zero.

3. Papéis de Lucas, ChatGPT, Antigravity e Codex

3.1 Lucas

Lucas decide produto, prioridade, escopo, visual final, aceite manual e autorização de commit/push. Toda mudança sensível exige autorização clara de Lucas.

3.2 ChatGPT

ChatGPT organiza o raciocínio, identifica riscos, define escopo, gera prompts quando Lucas pedir, audita relatórios, confere coerência com este documento e impede que tarefas avancem sem evidência. ChatGPT não deve sugerir implementação direta em área sensível sem auditar e mapear impacto.

3.3 Antigravity

Antigravity é executor limitado. Executa apenas o escopo pedido. Não decide arquitetura, produto, segurança, Supabase, DS V2 ou refatoração ampla. Problemas fora do escopo devem ser relatados, não corrigidos automaticamente.

3.4 Codex

Codex pode fazer auditoria local, inventário, leitura de repositório, cruzamento de chamadas, busca de legados e análise de uso real. Quando o pedido for auditoria, Codex deve permanecer em somente leitura. Implementação por Codex também exige prompt fechado, autorização e relatório factual.

Fluxo obrigatório: Lucas explica -> ChatGPT organiza -> executor audita/implementa só o escopo -> executor relata -> Lucas cola relatório -> ChatGPT audita -> Lucas valida visualmente quando houver UI -> commit/push apenas com autorização explícita.

4. Ordem obrigatória dentro de cada escopo

A ordem abaixo é obrigatória. Ela vale para CPF, admin, carteirinha, home, auth, documentos, notificações, suporte, DS V2, Supabase, Play Console ou qualquer outra frente.

- 0. Custo zero - verificar custo financeiro, limite gratuito, egress ou dependência paga.
- 1. Escopo - definir exatamente o que entra e o que fica fora.
- 2. Auditoria somente leitura - mapear arquivos, telas, fluxos, dados, DS V2, legado, services, Supabase e riscos sem alterar nada.
- 3. Classificação modular - decidir se o trecho é local, compartilhado controlado, global DS V2, Campo Cadastral, helper, service ou feature.
- 4. DS V2, satélites oficiais e legado - usar o que existe; se faltar padrão reutilizável, evoluir a camada correta antes de improvisar.
- 5. UI/UX e responsividade - checar clareza, semântica, contraste, 360dp a 480dp, SafeArea, teclado e Android-first.
- 6. Segurança/LGPD/Play Console - mapear dados pessoais/sensíveis, logs, documentos, privacidade e declarações reais.
- 7. Supabase/CRUD/GRANTS - checar create/read/update/delete, RLS, policies, grants, RPCs, Edge Functions e permissões.
- 8. Implementação em microfrente - alterar apenas arquivos autorizados, com menor recorte seguro e reversível.
- 9. Fluidez local - revisar rebuilds, setState, streams, dispose, loading, scroll, cache local e performance de uso.
- 10. Egress - reduzir chamadas desnecessárias, select genérico, stream pesada, leituras repetidas e payload excessivo.
- 11. Validação - executar validações proporcionais: analyze, build quando aplicável, diff, status, teste/manual.
- 12. Relatório, aceite e documentação viva - relatar evidências, riscos, pendências, status Git, validação visual e atualizar documentação permanente quando aplicável.

5. Auditoria, relatório e evidências

Auditoria somente leitura significa não alterar nada. Se qualquer arquivo novo, modificado ou removido aparecer no status final sem estar no status inicial, a auditoria deixou de ser somente leitura.

5.1 Auditoria obrigatória antes de implementar

- Ler código real; documentação antiga pode estar desatualizada.
- Identificar arquivos da tela/fluxo, widgets, helpers, services, models, routes e dependências.
- Mapear DS V2 usada, Campos Cadastrais, lacunas, legados ativos, hardcoded visual e duplicações.
- Mapear dados pessoais, dados sensíveis, Supabase, Auth, Storage, RLS, GRANTS, RPCs, Edge Functions, Drive/GAS, OneSignal e logs.
- Separar fato, inferência, risco, pendência futura e recomendação.
- Informar git status inicial e final.

5.2 Relatório bom

Relatório bom é evidência. Deve trazer objetivo, status Git inicial, comandos, arquivos lidos, arquivos alterados/criados/removidos, resumo do diff, validações, riscos, limitações, status Git final e confirmação de que não houve stage, commit ou push quando não autorizado.

5.3 Relatório ruim

Relatório ruim diz apenas 'feito', 'ajustado' ou 'sucesso' sem evidências. Se faltar status Git, arquivos lidos, arquivos alterados, validações, riscos ou limitações, o trabalho não deve ser aceito sem complemento.

Regra de aceite: ChatGPT não deve aceitar relatório genérico. Lucas não deve autorizar commit se o relatório não prova escopo, segurança, validação e estado do Git.

6. Camada Padronizada V2, DS V2 viva e satélites oficiais

A DS V2 é viva, expansiva e é a base de criação do ConeCTEA. Ela não é apenas uma pasta visual: representa a camada padronizada reutilizável do projeto. Isso inclui tokens, componentes, padrões visuais, campos, helpers globais e serviços utilitários que nasceram para evitar repetição e manter consistência.

6.1 DS V2 oficial

- lib/core/design_system_v2/ é o núcleo oficial.
- Tudo que nasce como componente, token ou padrão visual global deve preferencialmente usar prefixo Ds e morar dentro da DS V2.
- design_system_v2.dart é o barrel oficial. Novas telas devem preferir o barrel quando possível.
- Antes de usar qualquer item da DS V2, conferir a API real no código atual. Não inventar parâmetros, variantes ou nomes.
- Se a documentação e o código divergirem, o código real prevalece para implementação, mas a divergência deve gerar atualização documental.

6.2 DS V2 oficial expandida e satélites V2

Algumas pastas nasceram com a padronização V2, mesmo que historicamente não tenham seguido o prefixo Ds. Elas fazem parte integral do ecossistema V2 e devem ser tratadas como padrões oficiais, não como legado.

- lib/core/campos_cadastrais/ - satélite oficial da DS V2. Campos como CPF, e-mail, telefone, estado, cidade, gênero, raça/cor, tipo sanguíneo, CID e variantes protegidas devem ser usados nos fluxos correspondentes antes de qualquer TextField manual.
- lib/core/servicos/localizacao/ - satélite oficial de padronização global de localização, incluindo uso de IBGE quando aplicável. Não é visual, mas é padrão reutilizável do projeto.

- Helpers globais futuros devem nascer em local claro, com nome claro e rastreabilidade de versão/padrão, para que uma futura V3 consiga identificar o que nasceu na V2.
- Documentação de DS V2 e Campos Cadastrais é auxiliar. O Documento Central é a regra superior operacional; docs específicos podem detalhar APIs, inventários e exemplos.

6.3 Inventário operacional DS V2 v4.3

O inventário abaixo registra itens relatados como reais na auditoria local v4.3. Em qualquer frente, o executor deve conferir a API real antes de usar. Não se deve inventar componente, parâmetro ou enum por memória.

Tokens e estruturas visuais atuais: DsCores, DsCorVisual, DsEspacamentos, DsRaios, DsTamanhos, DsSombras, DsTipografia, DsTokenStatus e DsPaletasAvatar, quando presentes no código real.

Componentes e padrões DS V2 atuais: DsBotao, DsBotaoVoltar, DsCard, DsInput, DsDropdown, DsSearchableDropdown, DsMolduralcone, DsSelo, DsSeloStatus, DsSeloCargo, DsCardHub, DsCardNotificacao, DsMembrosCarrossel, DsMiniCarteiraPreview, DsCardFiltroContador, DsLoadingOverlay, DsLoadingSpinner, DsFeedback, DsStatusDialog e DsDialog, quando presentes no código real.

Feedback, loading e diálogo: DsFeedback.showSnackBar deve substituir SnackBar cru quando resolver o caso; DsLoadingSpinner deve substituir CircularProgressIndicator cru quando existir e couber; DsLoadingOverlay deve ser usado para operação em andamento que bloqueia interação; DsStatusDialog deve ser usado para retorno estruturado de sucesso/erro/alerta; DsDialog deve ser usado para confirmação/decisão quando sua API real atender ao fluxo.

Campos Cadastrais relevantes: CampoNomeCompleto, CampoNomeSocial, CampoCpf, CampoEmail, CampoTelefone, CampoDataNascimento, CampoEstado, CampoCidade, CampoGenero, CampoRacaCor, CampoTipoSanguineo, CampoCid, variantes protegidas e helpers/formatadores/validadores, quando presentes no código real.

6.4 Pergunta obrigatória antes de criar algo

- ☐ O que estou fazendo agora já existe na DS V2, Campos Cadastrais, localização/IBGE ou helper oficial?
- ☐ Existe parcialmente na DS V2? Se sim, devo completar/evoluir a DS V2 antes de criar solução solta?
- ☐ Isso pode ser reutilizado em outra tela, fluxo ou feature?
- ☐ Isso deve ser local, compartilhado controlado, global DS V2, Campo Cadastral ou helper oficial?
- ☐ Estou usando legado apenas porque é mais rápido?
- ☐ Estou criando uma coisa solta que futuramente vai obrigar manutenção manual tela por tela?
- ☐ Se criei/evolui componente, campo, helper, integração ou regra permanente, atualizei a documentação ao fim do escopo aprovado?

7. Legado, lacuna e regra de migração

7.1 Legado não é referência de futuro

Legado tolerado é aquilo que ainda precisa existir porque telas antigas dependem dele. Ele fica ativo até a migração dessas telas, mas não pode ser reutilizado em nova tela, refatoração nova ou componente novo.

7.2 Legado proibido para novos usos

- Tudo que começa com Premium em camada visual ou padronizável é legado proibido para novo uso.
- Tudo visual ou padronizável que começa com Conectea é legado proibido para novo uso, salvo exceção explicitamente reclassificada por auditoria de Lucas.
- AppColors, AppTextStyles, AppSpacing, AppRadius, AppShadows, StatusVisualTokens, ConecteaVisualTokens e caminhos paralelos de cor/estilo não devem ser usados em novas telas.
- PremiumAuthBackground, AppBackground, LoadingShimmer e equivalentes antigos não devem ser usados em novas frentes; se a frente precisar disso, criar/evoluir a DS V2 primeiro, salvo frente de manutenção em tela antiga com escopo travado.
- InputDecoration, TextStyle, Color(0x...), LinearGradient, BoxShadow e ButtonStyle hardcoded em tela nova só podem existir quando for caso local auditado e justificado. O padrão é usar DS V2.

7.3 Regra para lacuna DS V2

Faltou na DS V2? Se uma frente precisa de background, loading, skeleton, erro, empty state, auth background, scaffold, upload, bottom sheet ou outro padrão ainda inexistente, a primeira tarefa é evoluir a DS V2. Não reutilizar legado para ganhar velocidade.

Exemplo: se uma tela precisa de background, não usar AppBackground ou PremiumAuthBackground em nova frente. Criar DsBackground, DsScaffold, DsAuthBackground ou o nome tecnicamente adequado dentro da DS V2, usar no escopo atual e migrar telas antigas quando elas entrarem em escopo.

7.4 Legado morto, removido ou proibido

Legado morto é arquivo órfão ou classe sem uso real. Ele não deve ficar no projeto por apego histórico. Deve ser removido em microfrente segura, com auditoria de uso, git status, diff e validação.

Se um componente, helper, arquivo ou padrão legado for removido oficialmente, a nova versão deste Documento Central deve registrar essa remoção. A partir desse registro, qualquer ocorrência futura do item removido deve ser tratada como erro de manutenção: não reintroduzir, não copiar, não usar e migrar imediatamente dentro de escopo seguro.

8. Três camadas visuais: local, compartilhada e global

A camada visual do ConeCTEA deve ser classificada antes de criar, mover ou promover widgets. A regra evita dois erros: transformar a DS V2 em depósito de qualquer widget específico, ou duplicar componentes iguais em várias telas.

8.1 Camada local de feature/tela

- Usado em uma tela ou em uma composição muito específica.
- Depende de regra, dado, model, status ou contexto de uma feature.
- Pode ficar em features/.../widgets ou perto da tela.
- Exemplo: composições específicas da carteirinha digital, como frente/verso/background visual da carteirinha.

8.2 Camada compartilhada controlada

- Usado por duas ou três telas, normalmente do mesmo domínio.
- Tem o mesmo visual, os mesmos dados, o mesmo comportamento, a mesma regra de loading/erro e a mesma fonte/helper de dados.
- Se uma tela muda dados, layout, estado, fonte de consulta ou comportamento, deixa de ser compartilhado e volta a ser local.
- A preferência é ficar dentro da feature dona do conceito, por exemplo features/carteirinhas/shared/, evitando shared global virar depósito.

8.3 Camada global / DS V2

- Usado em muitas telas ou features, ou claramente esperado como padrão de produto.
- Não depende de regra específica de uma feature.
- Pode ser componente, token, padrão, helper global ou serviço utilitário padronizado.
- Existem globais idênticos e globais estruturais. DsCard e DsBotao são estruturais: a base é igual, o conteúdo muda.

Regra de promoção: uma tela = local. Duas ou três telas com uso exatamente igual = compartilhado controlado. Várias telas/features ou linguagem do app = global/DS V2. Se ganhou variações demais, reavaliar: DS parametrizada ou widgets locais separados.

9. Semântica visual, DsCores e DsTokenStatus

No ConeCTEA, cor é comunicação, mas nunca comunica sozinha. A intenção visual nasce da combinação entre cor, ícone, texto, forma, posição, contexto, componente e consequência da ação.

9.1 Caminho único de cores

- DsCores é o caminho único para cores estruturais e intenções visuais.
- DsCorVisual representa área, ação ou contexto visual.
- DsTokenStatus é separado e deve ser usado somente para status de carteirinha ou solicitação.
- Não recriar DsTokensVisuais, DsTokenVisual, ConecteaVisualTokens ou qualquer caminho paralelo de cores semânticas sem decisão explícita.
- Status persistido é uma coisa; intenção visual é outra.

9.2 Regra de composição visual

Composição obrigatória: intenção visual = cor + ícone + texto + contexto + consequência da ação. Cor sozinha não basta.

- Dados protegidos: texto claro, ícone coerente, token de dados protegidos/privacidade e explicação de análise quando houver.

- Solicitar correção ou alteração: intenção de revisão/correção, geralmente âmbar/laranja controlado, sem parecer erro destrutivo.
- Excluir/remover: intenção de perigo, vermelho controlado, texto explícito e confirmação.
- Carteirinha: área/produto usa intenção visual; status formal usa DsTokenStatus.

9.3 Regras cromáticas essenciais

- Fundo principal deve permanecer Night Blue / Dark Glass Premium.
- Cards devem ser neutros; cor semântica aparece em detalhe: ícone, borda, selo, faixa, glow sutil ou CTA.
- Não pintar card inteiro com cor do módulo sem justificativa.
- Contraste vem antes de estética.
- Evitar muitas cores fortes, excesso de glow, verde para qualquer ação e vermelho para pendência simples.

10. UI/UX neurodivergente e responsividade global

A interface do ConeCTEA deve funcionar para pessoas autistas, pessoas com TDAH, outras neurodivergências, familiares, responsáveis, administradores e usuários com baixa familiaridade digital. O objetivo não é estética 'para neurodivergentes', mas experiência clara, previsível, confortável e segura.

10.1 Princípios de UI/UX

- Clareza antes de ornamentação.
- Previsibilidade antes de surpresa.
- Conforto antes de densidade.
- Legibilidade antes de impacto visual.
- Segurança emocional antes de efeito bonito.
- Ação clara antes de muitas opções.
- Linguagem simples antes de texto técnico.
- Consistência antes de criatividade solta.

10.2 Responsividade Android-first

- Chrome é fallback visual inicial, não validação principal.
- Pensar em Android real, principalmente 360dp a 480dp.
- 360dp é o teste mental mínimo; em caso de dúvida, priorizar 360dp.
- Considerar SafeArea, status bar, navigation bar, 3 botões, gestos, teclado aberto, notch, cantos arredondados, fonte ampliada e aparelhos simples.
- Não resolver tela estreita encolhendo tudo; reorganizar, quebrar em blocos, permitir scroll seguro ou mover detalhes para tela/modal.

10.3 Proibições visuais recorrentes

- Texto pequeno para informação essencial.
- Ellipsis em motivo de reprova, prazo, erro, pendência, instrução ou dado necessário para agir.
- Botão esmagado, texto longo em selo, Row sem Flexible/Expanded, altura fixa perigosa.
- MediaQuery usado para calcular largura global quando o espaço real é local.
- Grid em tela estreita quando lista vertical seria mais legível.
- Brilho ou animação competindo com informação sensível.

11. Arquitetura modular por features

O ConeCTEA usa arquitetura modular por features. Modularidade é obrigatória para evitar telas gigantes, arquivos com milhares de linhas, mistura de visual com regra sensível e manutenção impossível.

11.1 Antes de crescer uma tela

- [] O trecho pertence à tela, feature, camada compartilhada, DS V2, Campos Cadastrais, service, model ou migration?
- [] O arquivo está ficando grande demais para auditar?
- [] Existe bloco visual repetido ou regra local que merece extração?
- [] A extração reduz risco ou cria complexidade desnecessária?
- [] Existe dado sensível, Supabase, Auth, permissão ou documento misturado com UI?
- [] Daria para modularizar agora em microrecorte seguro?

11.2 Modularizar cedo, mas sem refatorar tudo

- Tela coordena fluxo, estado local e composição principal; não deve guardar tudo.
- Widget de feature guarda composição específica e pode usar DS V2 por dentro.
- Helper de feature guarda formatação/cálculo local leve; não deve esconder regra crítica.
- Service guarda integração/regra de acesso; se envolver dados sensíveis, auditar melhor.
- DS V2 guarda padrão global reutilizável, sem regra de negócio sensível.

12. Segurança, LGPD, Play Console e dados sensíveis

Segurança não é etapa final. Ela orienta desenho, texto, fluxo, armazenamento, logs, permissão, documentação e publicação. Se envolver dado pessoal ou sensível, desacelerar e auditar.

12.1 Dados de cuidado obrigatório

- CPF, e-mail, telefone, data de nascimento, cidade/estado e dados de conta.
- Dependentes, crianças, adolescentes, responsáveis, vínculo familiar e contato de emergência.
- Documentos com foto, laudos, CID, tipo sanguíneo, raça/cor e informações relacionadas a neurodivergência/deficiência.
- QR Code, TEA ID, identificadores internos, status, motivos de pendência, recusa, suspensão e dados administrativos.
- Permissões, cargos, payloads, fileId, URLs de Drive, tokens, chaves e logs.

12.2 Regras de logs e exposição

- Não logar CPF completo, e-mail completo sem necessidade, documento, laudo, CID, fileId, URL completa, QR bruto, token, chave, payload sensível ou dado administrativo desnecessário.
- Mascarar dados em tela quando possível.
- Não colocar CPF em URL, nome de arquivo exposto, notificação, analytics ou logs.
- Não prometer segurança, exclusão, criptografia, anonimização ou descarte se o código real não prova isso.
- Política de Privacidade, Termos, Data Safety e app real precisam contar a mesma verdade.

12.3 Play Console

Políticas do Google Play podem mudar. Sempre que a frente envolver publicação, Data Safety, Health Apps, política de privacidade, declaração de saúde, permissão Android ou resposta ao Play Console, consultar fonte oficial atual e auditar o código real antes de responder.

13. Supabase, CRUD, RLS, GRANTs, RPCs e Edge Functions

Supabase é núcleo sensível do ConeCTEA. RLS, GRANTs, RPCs, Edge Functions, Storage, Auth, roles, realtime, validade de carteirinha e prazos devem ser tratados como área crítica.

13.1 Antes de tocar banco ou service

- ☐ Qual tabela, view, RPC, trigger, Edge Function, policy, GRANT ou service participa do fluxo?
- ☐ Quem pode criar, ler, atualizar e apagar cada dado?
- ☐ Existe dado pessoal/sensível?
- ☐ Existe risco de enumeração, vazamento, update amplo, N+1, stream pesada ou permissão excessiva?
- ☐ O client realmente precisa desses campos?
- ☐ A regra depende do servidor/banco e do fuso America/Sao_Paulo?
- ☐ Há migration reversível e pequena?
- ☐ Lucas autorizou explicitamente?

13.2 Regras de RPC, RLS e GRANTs

- RLS/GRANT/policy não devem ser alterados para silenciar alerta sem entender desenho real.
- Tabelas privadas com RLS ligado e sem policy podem estar corretas se o acesso for mediado por RPC segura.
- Função SECURITY DEFINER exige justificativa, search_path explícito, auth.uid()/ownership quando aplicável e grants restritos.

- Funções trigger-only não devem ficar executáveis diretamente por anon/authenticated/PUBLIC quando isso não for necessário.
- RPC chamada pelo app não pode ter grant revogado sem validar fluxo completo.
- Evitar select(*) e payload amplo; consultar apenas campos necessários.

13.3 Dívida técnica acoplada ao escopo

Se a frente atual mexe em CPF e existe dívida em RPC de CPF, validação de CPF, GRANT de CPF, enumeração por CPF ou log de CPF, essa dívida entra no escopo após auditoria e autorização. Se a dívida está em OneSignal ou pg_net e não afeta CPF, ela é registrada para frente futura.

14. Fluidez local e otimização de egress

Otimização de tela e otimização de egress são coisas diferentes. As duas são obrigatórias no fechamento de escopo, mas devem ser analisadas separadamente.

14.1 Fluidez local

- Foco na experiência da pessoa usando o app.
- Revisar rebuilds desnecessários, setState amplo, StreamSubscription, dispose, controllers, listeners, memory leak, loading/empty/error states, scroll e responsividade.
- Evitar streams no build e listeners sem cancelamento no dispose.
- Garantir feedback previsível, loading que não pula layout e tela que continua usável em aparelho simples.

14.2 Egress e consumo de banco

- Foco em não chamar banco toda hora sem necessidade e não gastar egress inútil.
- Evitar select(*) quando a tela usa poucos campos.
- Evitar streams pesadas, leituras repetidas, histórico infinito, payload grande e N+1 no client.
- Usar limit, paginação, cache local, RPC leve ou consulta agregada quando fizer sentido e for seguro.
- Otimizar egress sem esconder dado necessário nem quebrar segurança.

Regra prática: Fluidez local responde: a tela é confortável, rápida e previsível para a pessoa? Egress responde: o banco está sendo chamado apenas quando precisa, com o menor payload seguro?

15. Documentação, validação, commit e checklist final

15.1 Documentação

A documentação deve acompanhar o projeto, mas o código real prevalece quando houver conflito. Documentar não deve virar desculpa para alterar README, docs, scripts ou arquivos de referência sem autorização. Ao final de frentes maiores, registrar decisões técnicas, riscos, padrões novos e migrações realizadas.

A partir da v4.3, a atualização do Documento Central Obrigatório passa a fazer parte do fechamento institucional de escopos que criem, removam ou reclassifiquem padrões permanentes. Não é necessário atualizar este PDF após toda microcorreção pequena, mas é obrigatório após frente aprovada que altere regra de produto, DS V2, Campos Cadastrais, satélite V2, arquitetura, segurança, Supabase, Play Console, custo, integração ou legado.

15.2 Validações técnicas

- Microfrentes pequenas podem ter validação proporcional, mas fechamento de tela/fase exige validação completa.
- Alteração Dart/Flutter: flutter analyze como piso mínimo.
- Fechamento relevante: flutter build apk --debug quando aplicável.
- Sempre que houver alteração: git diff --check, git diff --name-only, git diff --stat e git status --short no relatório.
- Build limpo não prova UI boa. Analyze limpo não prova segurança. Validação técnica é piso, não aprovação final.

15.3 Commit e push

- Commit só depois de escopo conferido, relatório auditado, validações limpas e autorização explícita de Lucas.
- UI exige validação visual/manual de Lucas antes de commit quando a frente for visual.
- Lucas faz staging, commit e push manualmente pelo GitHub Desktop.
- Antigravity e Codex não podem fazer staging, commit ou push.
- ChatGPT não deve pedir git add ., git add -A, git commit ou git push como caminho padrão do projeto.
- Commit autorizado não significa push autorizado. Push também é decisão manual de Lucas no GitHub Desktop.

- Se git status estiver sujo por arquivos não explicados, não commitar.
- Depois de sugerir mensagem de commit, ChatGPT deve aguardar Lucas confirmar que comitou. Sem confirmação, o estado correto é commit sugerido, mas não confirmado.

15.4 Título e descrição de commit para GitHub Desktop

Quando uma frente estiver aprovada para commit, ChatGPT deve fornecer a Lucas o título e a descrição do commit prontos para copiar no GitHub Desktop. A mensagem deve ser curta, rastreável, segura e sem dados sensíveis.

Formato obrigatório: Título do commit; Descrição do commit; Observação para Lucas revisar os arquivos no GitHub Desktop antes de confirmar commit e push.

- Título: curto, claro e rastreável. Evitar genéricos como ajustes, melhorias, update ou várias coisas.
- Descrição: explicar escopo, o que mudou, áreas/arquivos principais, validações executadas e pendências conhecidas.
- Não incluir CPF, e-mail, nome de pessoa, protocolo real, ID interno, fileId, URL completa, QR bruto, token, chave, payload ou dado sensível.
- Preferir prefixos como docs, ds, ui, fix, security, supabase, auth, account, cpf, admin, perf, egress, refactor ou chore.
- Se houver pendência visual, risco residual ou validação não executada, isso deve aparecer na descrição antes de Lucas decidir commitar.

15.5 Checklist final obrigatório

- ☐ Escopo atual foi respeitado?
- ☐ Custo zero foi respeitado?
- ☐ Auditoria inicial foi feita?
- ☐ DS V2 e satélites oficiais foram usados/evoluídos quando necessário?
- ☐ Legado não foi reutilizado indevidamente?
- ☐ Widgets foram classificados como local, compartilhado controlado ou global?
- ☐ Modularidade foi aplicada sem refatoração ampla?
- ☐ UI/UX neurodivergente e semântica visual foram consideradas?
- ☐ Responsividade 360dp a 480dp foi considerada?
- ☐ Dados pessoais/sensíveis foram mapeados e protegidos?
- ☐ Supabase, CRUD, RLS, GRANTS, RPCs e logs foram avaliados quando aplicável?
- ☐ Fluidez local foi revisada?
- ☐ Egress foi revisado?
- ☐ Validações técnicas foram executadas ou justificadas?
- ☐ Relatório trouxe evidências reais?
- ☐ Lucas validou visualmente quando havia UI?
- ☐ Não houve staging, commit ou push pelo executor?
- ☐ ChatGPT forneceu título e descrição de commit apenas se a frente estiver pronta para Lucas decidir?
- ☐ Pendências futuras foram registradas sem invadir escopo?
- ☐ Se a frente criou regra permanente, a documentação central foi atualizada ou a atualização foi explicitamente agendada como próximo escopo?

Fechamento: se qualquer item obrigatório não passou, a frente não está fechada. Corrigir, auditar novamente ou registrar pendência conforme o risco. Só iniciar o próximo escopo depois do fechamento consciente do atual.

16. GitHub Desktop: commit e push manuais

Esta seção integra definitivamente a regra de commit/push manual ao corpo principal do documento. Ela deve ser lida antes de qualquer fechamento de frente.

Regra principal: Lucas é a única pessoa que realiza staging, commit e push no fluxo normal do ConeCTEA, usando o GitHub Desktop. ChatGPT fornece mensagem segura; Antigravity e Codex apenas relatam evidências.

16.1 O que cada papel faz

- Lucas revisa os arquivos no GitHub Desktop, escolhe o que entra no commit, escreve/cola título e descrição, confirma o commit e decide se fará push.
- ChatGPT audita o relatório, identifica se a frente pode ser commitada e fornece título/descrição seguros quando fizer sentido.
- Antigravity/Codex não executam commit, push, staging, git add, git commit, git push ou comandos equivalentes sem autorização explícita e excepcional de Lucas.
- Relatório de executor deve informar claramente se não houve staging, commit ou push.

16.2 Modelo de mensagem de commit

Modelo: Título do commit: <tipo>: <resumo curto do escopo> | Descrição: o que mudou, áreas/arquivos principais sem expor dado sensível, validações executadas e pendências conhecidas | Observação para Lucas: revisar arquivos no GitHub Desktop antes do commit e fazer push manualmente apenas se esse for o próximo passo autorizado.

16.3 Regra de não assumir commit

Depois de fornecer título e descrição, ChatGPT deve aguardar Lucas informar que realizou o commit pelo GitHub Desktop. Enquanto Lucas não confirmar, o estado correto é: commit sugerido, mas não confirmado. Push também deve ser confirmado por Lucas quando for relevante ao próximo passo.

17. Governança de atualização do documento central

O Documento Central Obrigatório é uma fonte viva e versionada do Projeto ConeCTEA. Ele pode e deve ser atualizado quando novas regras permanentes forem decididas durante o percurso, especialmente após escopos grandes, auditorias relevantes, evoluções da DS V2, mudanças de segurança, mudanças de arquitetura, novas exigências do Play Console ou decisões de produto.

Regra de preservação: nada que já foi definido como regra obrigatória deve ser apagado ou enfraquecido sem registro claro de substituição. A forma correta de evoluir é adicionar regra nova, registrar versão nova e indicar quando a regra nova complementa, substitui ou restringe a regra anterior.

17.1 Quando atualizar este documento

- Depois de escopo grande que criou regra permanente de produto, DS V2, segurança, fluxo, modularidade ou validação.
- Quando a DS V2 ganhar novo componente, token, padrão, helper global ou regra semântica que deve ser lembrada em novos chats.
- Quando Campos Cadastrais, localização/IBGE ou outro satélite V2 ganhar novo campo, serviço, helper ou regra permanente.
- Quando uma lacuna deixar de ser lacuna porque virou DS V2 oficial ou satélite oficial.
- Quando uma regra de legado mudar de tolerado para morto/removível ou de tolerado para proibido.
- Quando um componente/padrão legado for apagado oficialmente; nesse caso, registrar que ele não existe mais e que qualquer uso futuro deve ser migrado imediatamente em escopo seguro.
- Quando um novo risco de custo, egress, Play Console, LGPD, Supabase, Git ou arquitetura for descoberto.
- Quando uma decisão de Lucas precisar virar regra permanente para todos os próximos escopos.

- Quando o escopo exigir algo não previsto no documento e essa expansão for aprovada para atingir o resultado de produto.

17.2 Como atualizar

- [] Criar nova versão numerada do PDF, mantendo histórico.
- [] Informar o que foi adicionado, o que foi refinado, o que foi removido/reclassificado e por quê.
- [] Não esconder regra nova em local difícil quando ela muda fluxo central; integrar no corpo principal na próxima versão completa.
- [] Manter o documento mais novo como fonte fixada principal do Projeto.
- [] Se houver conflito entre versões, prevalece a versão mais nova quando ela for mais segura, mais restritiva, mais gratuita, mais acessível ou mais alinhada ao produto final.
- [] Atualização de documento não autoriza alteração de código. Código só muda por escopo próprio, com auditoria e prompt autorizado.

17.3 O que pode ser atualizado

- Inventário da DS V2, incluindo componentes, tokens, padrões, Campos Cadastrais, localização e helpers padronizados.
- Regras de semântica, semiótica, DsCores, DsCorVisual e DsTokenStatus.
- Classificação de legado tolerado, legado proibido, legado morto e legado removido.
- Regras de modularidade local, compartilhada controlada e global.
- Regras de responsividade, Android-first, validação visual e acessibilidade.
- Regras de segurança, dados sensíveis, Play Console, Supabase, CRUD, RLS, GRANTS, RPCs, Edge Functions e egress.
- Regras de custo zero e alternativas gratuitas.
- Regras de GitHub Desktop, título/descrição de commit e confirmação manual de push.

17.4 Fechamento de escopo e atualização obrigatória

Ao final de uma frente importante, ChatGPT deve perguntar se alguma decisão tomada virou regra permanente. Se sim, a próxima ação correta é atualizar este Documento Central Obrigatório em nova versão, sem apagar o histórico e sem enfraquecer regras já estabelecidas.

Exemplo: se durante o escopo de dependentes foi necessário criar ou oficializar novo componente DS, novo Campo Cadastral, novo helper, nova regra de CPF, nova RPC padrão, nova validação visual ou remoção de legado, isso deve entrar na próxima versão do Documento Central após auditoria final e aceite da frente.

18. Frases de decisão para novos chats

Ao iniciar um novo chat ou nova frente, ChatGPT deve aplicar estas perguntas antes de gerar qualquer prompt:

- [] Qual é o escopo exato autorizado por Lucas?
- [] Essa frente toca dado sensível, Supabase, Auth, RLS, migration, Play Console ou custo externo?
- [] O que existe hoje no código real?
- [] O que a DS V2, Campos Cadastrais e satélites oficiais já cobrem?
- [] Existe legado no caminho? Ele é tolerado, proibido, morto ou lacuna?
- [] O trecho deve ser local, compartilhado controlado ou global?
- [] Há dívida técnica acoplada ao escopo que deve ser resolvida agora?
- [] A solução continua 100% gratuita?
- [] A tela funciona em Android 360dp a 480dp?
- [] O banco está sendo usado com o menor payload seguro?
- [] Qual é o menor prompt seguro para o executor?
- [] Se o escopo evoluir regra permanente, como a documentação central será atualizada no fechamento?

19. Resumo executivo obrigatório

Custo: ConeCTEA deve funcionar com custo zero. Recurso pago não entra.

Escopo: Cada frente cumpre todas as etapas antes da próxima.

Auditoria: Antes de implementar, auditar código real em somente leitura.

DS V2: Base viva e expansiva. Faltou padrão reutilizável? Evoluir DS V2 primeiro.

Satélites V2: Campos Cadastrais, localização/IBGE e helpers oficiais fazem parte do ecossistema V2.

Legado: Tolerado só para manter tela antiga funcionando; proibido para novo uso.

Legado removido: Se foi removido oficialmente, qualquer uso futuro é erro e deve ser migrado.

Modularidade: Não deixar arquivo virar gigante; modularizar em microfrente segura.

Compartilhado: Só é compartilhado se for exatamente igual em 2 ou 3 telas.

Global: Virou linguagem do app ou muitas telas: DS V2/global.

Segurança: Dados pessoais/sensíveis exigem auditoria, minimização, logs seguros e verdade documental.

Supabase: RLS, GRANTS, RPCs, Edge e Auth exigem autorização e plano reversível.

UI: Clareza, previsibilidade, contraste, baixa carga cognitiva e Android-first.

Fluidez: Experiência local: rebuild, setState, streams, dispose, loading, scroll.

Egress: Banco: menos chamadas, menos payload, menos stream pesada, sem select(*) desnecessário.

GitHub Desktop: Lucas faz staging, commit e push manualmente. ChatGPT fornece título e descrição seguros.

Documento vivo: Novas regras permanentes entram em nova versão. Não enfraquecer regra antiga sem registro claro.

Fechamento: Relatório factual, validações, status Git, validação visual quando UI e commit só autorizado.

20. Fontes consolidadas, inventário v4.3 e histórico

Este documento central consolida os manuais internos, estudos técnicos e decisões de produto do ConeCTEA, incluindo operação Lucas + ChatGPT + Antigravity/Codex, auditoria e relatórios, evolução da DS V2, arquitetura modular por features, UI/UX neurodivergente, responsividade global, segurança/LGPD/Play Console, DsCores, semântica visual, semiótica e psicologia das cores.

A versão 4.3 mantém as regras obrigatórias da v4.2 e acrescenta/refina principalmente: governança de atualização obrigatória do Documento Central após escopos que criem regra permanente; inventário operacional DS V2 v4.3; reconhecimento explícito de Campos Cadastrais e localização/IBGE como satélites oficiais do ecossistema V2; regras para feedback, snackbar, loading e dialogs; e regra de registro de legado removido.

20.1 Histórico da versão v4.3

- Atualizada a regra de documento vivo para exigir atualização após fechamento de escopos que expandam capacidades permanentes do projeto.
- Registrado inventário operacional de tokens, componentes e padrões DS V2 relatados na auditoria v4.3.
- Incluídos DsLoadingOverlay, DsLoadingSpinner, DsFeedback.showSnackBar, DsStatusDialog e DsDialog quando confirmados pelo código real.
- Reforçada a integração dos Campos Cadastrais com a DS V2 como satélite oficial.
- Reforçada a relação com localização/IBGE como satélite V2 não visual.
- Criada regra explícita para componente legado removido: se foi apagado e documentado, qualquer uso futuro deve ser tratado como erro de manutenção.
- Reforçado que atualização documental não autoriza alteração de código; código só muda por escopo próprio.

20.2 Regra final

Quando houver diferença entre documentação, memória de conversa e código, o código real atual prevalece para implementação. Quando houver conflito entre regras, prevalece a opção mais segura, mais restritiva, mais gratuita, mais acessível e mais alinhada ao produto final.

Fim do Documento Central Obrigatório v4.3.